

ML-DL-OPS

The ML Pipeline

Data to Deployment: Stages, Challenges & Patterns

Quick Recap: Lecture 1

87% of ML projects never reach production

5% Model is 5% of work; production-readiness is 95%

MLOps = ML + Dev + Ops — bridging lab to production

Case Study: Our deepfake detector faced 10+ challenges going from lab to election-scale deployment

TODAY'S FOCUS

Deep Dive into the ML Pipeline

Each stage in detail

Challenges at every step

Technical debt in ML systems

Architecture patterns

Reading: "Hidden Technical Debt in ML Systems"

Quick Check-In

1. Git and Docker Setup

How many of you successfully installed Docker and created Git account?

2. The Google Paper

"Hidden Technical Debt in ML Systems"

3. Your Experience

Have you deployed any ML model before? What challenges did you face?

The Complete ML Pipeline



↺ Continuous Feedback Loop — Monitoring triggers retraining

DATA LAYER

Data Lakes, Feature Stores, Data Versioning (DVC), Data Validation

MODEL LAYER

Experiment Tracking, Model Registry, Model Versioning, Artifact Storage

INFRA LAYER

Containers, Orchestration, CI/CD, Compute (GPU/TPU), Serving Infrastructure

Key Insight: It's not just a linear pipeline — it's an interconnected system with feedback loops, versioning at every layer, and cross-cutting concerns (security, logging, testing).

Two Pipelines: Training vs Inference



TRAINING PIPELINE

Goal: Produce a trained model

Frequency: Periodic (daily/weekly/on-demand)

Latency: Hours to days acceptable

Data: Large historical batches

Compute: GPU clusters, high memory



Must be consistent!



INFERENCE PIPELINE

Goal: Serve predictions to users

Frequency: Continuous (24/7)

Latency: Milliseconds required

Data: Single requests, real-time

Compute: Optimized for throughput

Critical: Feature computation must be IDENTICAL in both pipelines. Different code = training-serving skew = silent failures.



Stage 1: Data Ingestion

WHAT HAPPENS HERE

Collect data from multiple sources

Handle different formats (CSV, JSON, images, video)

Batch vs streaming ingestion

Store raw data with metadata

DATA SOURCES

Databases, APIs, File uploads, IoT sensors, Web scraping, Third-party datasets, User-generated content

CHALLENGES

Scale: Terabytes/day, millions of files

Heterogeneity: Different formats, schemas, encodings

Reliability: Network failures, corrupt files, missing data

Privacy: PII handling, consent, GDPR compliance

Latency: Real-time vs batch trade-offs

TOOLS

Batch: Apache Spark, Airflow, Prefect

Streaming: Kafka, Flink, Kinesis

Storage: S3, GCS, HDFS, Delta Lake

Versioning: DVC, LakeFS

Remember our case study: JPG vs PNG vs HEIC — this is where that problem starts

When Data Ingestion Goes Wrong: Real Failures

UBER ATG (2018)

Incident: Self-driving car killed pedestrian Elaine Herzberg in Arizona

Data Issue: Training data only included pedestrians near crosswalks. System couldn't classify jaywalking pedestrians.

Result: Car cycled between "vehicle," "bicycle," "unknown" for 6 seconds before impact

Source: NTSB Report, Nov 2019

TESLA AUTOPILOT

Incident: Shadow mode detected "bicycles in middle of road" near cars

Data Issue: Training data didn't distinguish bikes ON car racks from bikes on road

Fix: Fleet query for similar cases → relabel → retrain

Source: Tesla AI Day 2019

AMAZON RECRUITING (2018)

Incident: AI recruiting tool penalized resumes with word "women's"

Data Issue: Trained on 10 years of male-dominated hiring data

Result: System learned historical bias, project scrapped

Source: Reuters, Oct 2018

APPLE CARD (2019)

Incident: Women received lower credit limits than spouses with same finances

Data Issue: Historical credit data reflected systemic gender bias

Result: Regulatory investigation, PR crisis

Source: Bloomberg, Nov 2019

Common Thread: All failures trace back to incomplete, biased, or poorly labeled training data — not model architecture. Data quality is life-or-death.

Data Contracts: The Missing Bridge

WHAT IS A DATA CONTRACT?

A formal agreement between data producers and consumers specifying:

- ✓ **Schema:** Columns, data types
- ✓ **Constraints:** Ranges, allowed values
- ✓ **Null rules:** Which fields can be null
- ✓ **Freshness SLA:** Data arrives by X time
- ✓ **Volume expectations:** ~N rows/day

EXAMPLE: IMAGE UPLOAD CONTRACT

```
contract: image_uploads_v2
owner: media-team
schema:
  - image_id: string (required)
  - format: enum [jpg, png, webp]
  - width: int (100-4096)
  - height: int (100-4096)
  - upload_time: timestamp (required)
  - user_id: string (nullable)
freshness: 15 minutes
breaking_change_policy: 7-day notice
```

BREAKING CHANGES

Without contracts: Upstream team changes column name → your pipeline breaks at 3 AM

With contracts: Change requires version bump, notification, migration period

Versioned schemas: v1 → v2 with backwards compatibility or clear migration

Labeling is Part of the Pipeline

LABEL COLLECTION

Annotation quality: Clear guidelines, trained annotators, edge case documentation

Inter-annotator agreement: Cohen's kappa, majority voting, adjudication process

Cost/time tradeoffs: In-house vs crowdsourced vs synthetic labels

Tools: Label Studio (recommended), Labelbox, Scale AI

LABEL QUALITY ISSUES

Label noise: Incorrect labels degrade model quality. Even 5% noise can significantly hurt performance.

Label drift: Definition of "positive" changes over time. What was spam in 2020 isn't spam in 2025.

Class imbalance: Real-world labels are rarely 50/50. Deepfakes are rare vs real content.

FEEDBACK LOOPS ⚠️

The Hidden Danger:

Model predictions influence future labels → model learns its own biases → amplification loop

Example: Recommendation system shows items → users click → clicks become training labels → system reinforces its choices

Mitigation: Exploration/randomization, holdout groups, human audits



Stage 2: Data Processing & Validation

PROCESSING TASKS

Cleaning: Handle missing values, remove duplicates, fix errors

Transformation: Normalize, resize, encode, format conversion

Augmentation: Synthetic data, oversampling, noise injection

Splitting: Train/val/test with stratification

DATA VALIDATION (CRITICAL!)

Schema validation: Expected columns, types, ranges

Statistical validation: Distribution checks, anomaly detection

Drift detection: Compare new data vs training data

Quality metrics: Completeness, consistency, accuracy

TOOLS

Validation: Great Expectations, TFDV, Pandera

Processing: Pandas, Polars, Spark, Dask

Image/Video: OpenCV, FFmpeg, Pillow

Golden Rule: Never trust incoming data. Validate everything. Fail fast.

"Garbage in, garbage out" — Most production failures trace back to data issues

Testing: The Cross-Cutting Pillar

DATA TESTS

Schema validation (columns, types)

Range checks (age > 0, price > 0)

Missingness thresholds

Duplication checks

Label leakage detection

Tool: Great Expectations

FEATURE TESTS

Point-in-time correctness

Offline/online parity checks

Feature distribution tests

Null handling consistency

Tool: Feast validation

MODEL TESTS

Regression tests (no metric degradation)

Calibration checks

Slice analysis (subgroup performance)

Robustness to perturbations

Tool: MLflow + pytest

PIPELINE TESTS

Unit tests (each component)

Integration tests (mocked data)

End-to-end tests (sample data)

Smoke tests post-deploy

Tool: pytest + GitHub Actions



Stage 3: Feature Engineering

FEATURE TYPES

Raw features: Direct from data (age, price, pixel values)

Derived features: Computed (ratios, aggregates, time deltas)

Embeddings: Learned representations (text, image, user)

Cross features: Interactions between features

FEATURE STORE (KEY CONCEPT)

A centralized repository for storing, managing, and serving features

Reusability: Same features across models

Consistency: Training = Serving (no skew)

Point-in-time: Historical feature values

Low latency: Online serving in ms

Tools: Feast, Tecton, Hopsworks, AWS Feature Store

TRAINING-SERVING SKEW

The Silent Killer:

Features computed differently in training vs production → model performs worse than expected

Causes: Different code paths, stale data, timing issues, library versions

Solution: Single feature computation pipeline for both training and serving



Stage 4: Model Training

TRAINING IN PRODUCTION

Distributed training: Multi-GPU, multi-node (DDP, FSDP)

Hyperparameter tuning: Grid, random, Bayesian, population-based

Checkpointing: Save progress, resume from failures

Resource management: GPU allocation, spot instances, preemption

EXPERIMENT TRACKING (ESSENTIAL)

Track everything for reproducibility:

Parameters: Hyperparameters, config

Metrics: Loss, accuracy, custom metrics

Artifacts: Models, plots, outputs

Code: Git commit, dependencies

Data: Dataset version, splits

Tools: MLflow, W&B, Neptune, CometML, TensorBoard

REPRODUCIBILITY

Can you reproduce results from 6 months ago?

Checklist:

- ☐ Fixed random seeds
- ☐ Pinned dependencies
- ☐ Versioned data
- ☐ Containerized environment
- ☐ Logged everything

Distributed Data Parallel (DDP) vs Fully Sharded Data Parallel (FSDP)

Quick Quiz: Check Your Understanding

Q1: Data Validation Failure

New data arrives but violates the schema (wrong column type). What should happen?

A) Log warning, continue processing B) Fail fast, block pipeline C) Auto-fix the data D) Skip bad rows silently

Q2: Training-Serving Skew

Your model has 95% accuracy in testing but only 80% in production. What's the most likely cause?

A) Model is overfitting B) Training-serving skew C) Not enough data D) Wrong architecture

Q3: What Caused Zillow's Failure?

Zillow lost \$500M on their iBuying ML system (Ref: Bloomberg, Nov 2021). What type of drift caused the failure?

A) Prediction drift B) Data drift C) Concept drift D) Feature drift

Discuss with your neighbor for 2 minutes, then we'll review together

Quiz Answers

Q1: Answer — B) Fail fast, block pipeline

Schema violations should halt processing immediately. "Garbage in, garbage out" — bad data leads to bad models. Auto-fixing is dangerous; silent skipping hides problems.

Q2: Answer — B) Training-Serving Skew

When test accuracy is high but production is low, features are likely computed differently in training vs serving. This is why feature stores and data contracts exist!

Q3: Answer — C) Concept Drift

The relationship between inputs and outputs changed — housing market dynamics shifted, but the model still predicted based on old patterns. Data drift = input distribution changes; concept drift = underlying relationship changes.

These are exactly the kinds of issues MLOps practices help you detect and address

Lineage & Metadata: Beyond Versioning

THE 3 AM QUESTION

"Production model started behaving differently.
What changed?"

Was it the data?

Was it the code?

Was it the config?

Was it a dependency update?

Was it upstream data source?

Without lineage: Hours of debugging. **With lineage:** Minutes to root cause.

LINEAGE TRACKS

Model v2.3 → produced by



Dataset: training_v5_2024-01-10



Code: git commit abc123



Config: lr=0.001, batch=32, epochs=50



Deps: torch==2.1.0, numpy==1.24.0



Env: Docker image sha256:def456

WHY IT MATTERS

✓ Auditability

✓ Reproducibility

✓ Debugging

✓ Compliance

✓ Impact analysis

Tools: MLflow, DVC, W&B, DataHub



Stage 5: Model Evaluation

BEYOND ACCURACY

Performance: Latency, throughput, memory footprint

Fairness: Bias across groups (gender, age, ethnicity)

Robustness: Adversarial inputs, edge cases, noise

Explainability: Why did the model decide this?

Cost: Inference cost per prediction

VALIDATION STRATEGY

Offline evaluation: Hold-out set, cross-validation

Shadow mode: Run in parallel with production, compare

A/B testing: Gradual rollout, statistical significance

Canary deployment: Small % traffic first

MODEL REGISTRY

Central catalog of trained models:

Version history

Stage: dev → staging → prod

Metadata & lineage

Approval workflows

Tools: MLflow Registry, SageMaker, Vertex AI

A model that's 95% accurate but takes 10 seconds per prediction is useless in production



Stage 6: Model Deployment

DEPLOYMENT PATTERNS

Batch inference: Periodic predictions on stored data

Real-time (online): Request-response via API

Streaming: Continuous predictions on data streams

Edge/On-device: Model runs on user device

SERVING INFRASTRUCTURE

Containerization: Docker for reproducibility

Orchestration: Kubernetes for scaling

Load balancing: Distribute requests

Auto-scaling: Handle traffic spikes

GPU sharing: Maximize utilization

Tools: TorchServe, TF Serving, Triton, BentoML, Seldon

RELEASE STRATEGIES

Blue-Green: Switch traffic between two environments

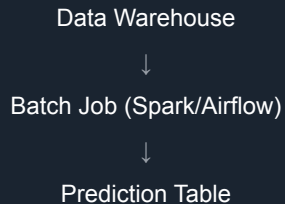
Canary: Gradual rollout (1% → 10% → 50% → 100%)

Rolling: Update instances incrementally

Always have rollback! Knight Capital lost \$440M in 45 min without one (SEC Report, 2013)

ML System Architecture Patterns

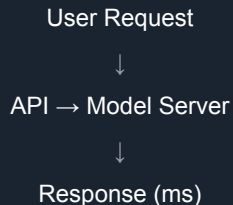
BATCH PREDICTION



Use when: Predictions needed periodically, latency not critical

Example: Nightly recommendation emails

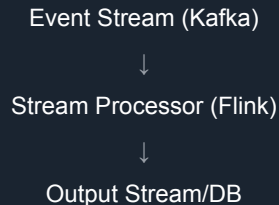
REAL-TIME (ONLINE)



Use when: Low latency required, user-facing

Example: Fraud detection, deepfake check

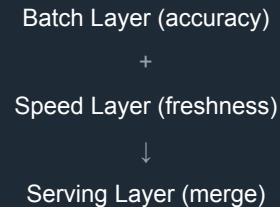
STREAMING



Use when: Continuous data, near-real-time

Example: Anomaly detection, live feeds

HYBRID (LAMBDA)



Use when: Need both accuracy & freshness

Example: Search ranking, feeds

SLOs & Error Budgets

DEFINITIONS

SLI (Indicator)

What you measure: latency, error rate, throughput

SLO (Objective)

Internal target: "p95 latency < 200ms"

SLA (Agreement)

External contract: "99.9% uptime or refund"

EXAMPLE: DEEPPFAKE API SLOs

Latency: p95 < 500ms for images < 5MB

Availability: 99.5% uptime (allows ~3.6 hrs/month downtime)

Error rate: < 0.1% failed requests

Cost: < \$0.001 per prediction (infra cost)

ERROR BUDGET

99.5% uptime = **0.5% error budget**
= ~3.6 hours/month allowed downtime

Budget remaining? Ship new features, experiment

Budget exhausted? Freeze deploys, fix reliability

Rollback trigger: If error rate > 1% for 5 min
→ auto-rollback

SLOs make "good enough" concrete and deployments data-driven rather than fear-driven



Stage 7: Monitoring & Observability

WHAT TO MONITOR

System metrics: CPU, GPU, memory, latency, errors

Model metrics: Prediction distribution, confidence scores

Data metrics: Input distribution, missing values, outliers

Business metrics: Conversion, engagement, revenue impact

DRIFT DETECTION (CRITICAL)

Data drift: Input distribution changes (new user types, seasonal patterns)

Concept drift: Relationship between input and output changes

Prediction drift: Output distribution shifts unexpectedly

Zillow lost \$500M because they didn't detect market drift (Bloomberg, Nov 2021)

TOOLS

Infra: Prometheus, Grafana, DataDog

ML-specific: Evidently AI, WhyLabs, Arize

Alerting: PagerDuty, Slack webhooks

Feedback Loop: Monitoring triggers → Alerts → Retrain → Redeploy

In our deepfake system, we detected adversarial attacks through monitoring anomalies

Observability: The Three Pillars



METRICS

Numeric measurements over time

Latency (p50, p95, p99)

Error rate, throughput

GPU utilization, memory

Drift statistics

Prediction distribution

Tool: Prometheus + Grafana



LOGS

Discrete events with context

Input/output sampling

Failure reasons, stack traces

Data validation failures

Model version loaded

Config changes

Tool: ELK Stack, CloudWatch



TRACES

Request flow across services

Feature fetch latency

Preprocessing time

Model inference time

Postprocessing time

Network overhead

Tool: Jaeger, OpenTelemetry

All three together answer: What happened? Why? Where was time spent?

Security & Privacy: Production Checklist

DATA SECURITY

- ☐ PII identification & masking
- ☐ Data encryption at rest
- ☐ Data encryption in transit (TLS)
- ☐ Data retention policies
- ☐ GDPR/DPDPA compliance

ACCESS CONTROL

- ☐ Role-based access (RBAC)
- ☐ API authentication (API keys, OAuth)
- ☐ Secrets management (Vault, AWS Secrets)
- ☐ No secrets in code/configs
- ☐ Audit logging of access

ENDPOINT SECURITY

- ☐ Rate limiting per client
- ☐ Input validation & sanitization
- ☐ Request size limits
- ☐ Adversarial input detection
- ☐ DDoS protection

LOGGING POLICY

- ✓ Log request metadata
- ✓ Log errors & stack traces
- ✗ Never log raw PII
- ✗ Never log API keys/tokens
- ✗ Never log full images (sample)

Case Study: Our Deepfake System Pipeline

INGESTION

API receives images/videos

Challenge: JPG, PNG, HEIC, MP4, WebM...

PROCESSING

Format conversion, resize to 256×256

Challenge: Codec handling, face detection

FEATURES

Face embeddings, frequency analysis

Challenge: Same extraction train/serve

TRAINING

Multi-GPU training on FF++, DFDC

Challenge: Batch size 32 vs single image

EVALUATION

Accuracy + latency + fairness across demographics

Challenge: Hindi vs English content

DEPLOYMENT

Challenge: 100K+ req/hr on election day

MONITORING

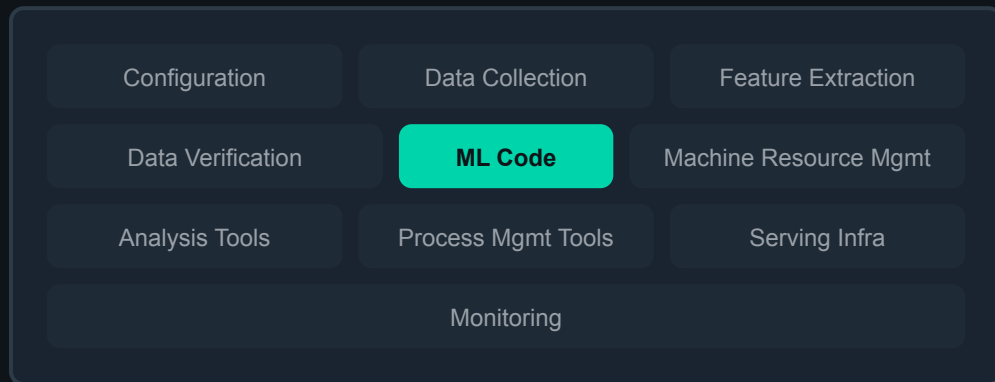
We detected **adversarial patterns**: repeated submissions with slight perturbations, unusual file metadata, traffic from suspicious IPs during election peaks. Monitoring saved us.

Lesson: Every pipeline stage had production challenges we never anticipated in research. The pipeline view helped us systematically address each one.

Technical Debt in ML Systems

From "Hidden Technical Debt in Machine Learning Systems" — Google, NeurIPS 2015

THE ML CODE IS THE TINY BOX



TYPES OF ML TECHNICAL DEBT

Entanglement: Changing Anything Changes Everything (CACE)

Hidden feedback loops: Model affects its own training data

Undeclared consumers: Unknown systems depend on your model

Data dependencies: Unstable, underutilized, legacy features

Configuration debt: Hyperparameters scattered everywhere

Pipeline jungles: Glue code connecting systems

This paper is a must-read for anyone building production ML systems

ML Pipeline Anti-Patterns (Avoid These!)

🚫 NOTEBOOK TO PRODUCTION

"Just export the notebook as a script"

Why bad: Hidden state, no tests, no error handling, hardcoded paths

🚫 NO DATA VERSIONING

"The data is in that folder somewhere"

Why bad: Can't reproduce results, can't debug issues

🚫 "IT WORKS ON MY MACHINE"

No containerization, unpinned dependencies

Why bad: Results not reproducible, debugging nightmare

🚫 SINGLE METRIC OBSESSION

"Just maximize accuracy"

Why bad: Ignores latency, fairness, cost, robustness

🚫 MANUAL MODEL UPDATES

"I'll just copy the new model to the server"

Why bad: No rollback, no audit trail, human error

🚫 DEPLOY AND FORGET

"Model is deployed, we're done!"

Why bad: Data drift, model decay, silent failures

Every anti-pattern we've seen in real projects. This course teaches you the right patterns.

MLOps Maturity Levels

0

Manual

Manual, script-driven

Notebooks to production

No versioning

Manual deployment

No monitoring

Where most research starts

1

ML Pipeline

Automated training pipeline

Experiment tracking

Model registry

Manual deployment trigger

Basic monitoring

This course target

2

CI/CD + CT

CI/CD for ML pipelines

Continuous Training (CT)

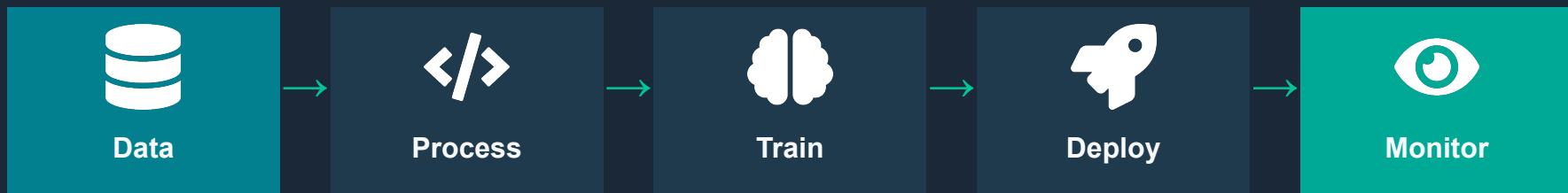
Automated retraining

Auto deployment

Drift-triggered updates

Production-grade systems

Let us build “Sentiment Classifier”



THE PROBLEM

- Classify movie reviews
- Binary: Positive/Negative
- IMDB dataset (50K reviews)

THE STACK

- Python + PyTorch
- FastAPI for serving
- MLflow for tracking
- Docker + GitHub Actions

WHY THIS PROJECT?

- Simple enough to finish
- Complex enough to learn
- All MLOps patterns apply



The Detailed ML Pipeline

Each stage has inputs, outputs, and tools — all must work together

1 Data Ingestion

In: Raw sources (APIs, files)

Out: Raw dataset (Parquet)

Tool: HuggingFace datasets

`src/data/loader.py`

2 Data Validation

In: Raw dataset

Out: Validated dataset

Tool: Great Expectations

`src/data/validate.py`

3 Data Processing

In: Validated dataset

Out: Clean train/val/test splits

Tool: Pandas, sklearn

`src/data/process.py`

4 Feature Engineering

In: Clean text data

Out: Tokenized tensors

Tool: Transformers tokenizer

`src/features/`

5 Model Training

In: Feature tensors + labels

Out: Trained model weights

Tool: PyTorch, MLflow

`src/models/train.py`

6 Evaluation

In: Model + test data

Out: Metrics, reports

Tool: sklearn metrics

`src/models/evaluate.py`

7 Model Registry

In: Trained model

Out: Versioned artifact

Tool: MLflow Registry

8 Serving / API

In: Model artifact

Out: REST predictions

Tool: FastAPI, Docker

9 Monitoring

In: Predictions, requests

Out: Metrics, alerts

Tool: Prometheus, logs

↻ Continuous Feedback Loop: Monitoring detects drift → Triggers retraining → Model updated → Redeployed

Project Structure: The Foundation

```
sentiment-classifier/
├── data/
│   ├── raw/
│   └── processed/
├── src/
│   ├── data/
│   ├── features/
│   ├── models/
│   └── api/
├── tests/
├── configs/
├── notebooks/
├── models/           # Saved models
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
├── pyproject.toml
└── README.md
```

data/

Raw & processed datasets (git-ignored, DVC tracked)

src/

Production code organized by pipeline stage

tests/

Unit, integration, and pipeline tests

configs/

YAML configs for experiments (versioned)

models/

Saved model artifacts (git-ignored)

Docker*

Containerization for reproducibility